

Лабораторная работа № 4

Имитационная модель TCP/Reno и активного управления очередью RED

Куокконен Дарина Андреевна

2026-08-30

Аннотация

Построена имитационная модель 25 конкурирующих потоков TCP/Reno, проходящих через узкое место 20 Мбит/с с активной очередью RED. Исследованы окно перегрузки, мгновенная и сглаженная длины очереди, вероятность раннего сброса и пропускная способность. Проведён параметрический эксперимент по числу потоков; реализация проверена автоматическими тестами.

Содержание

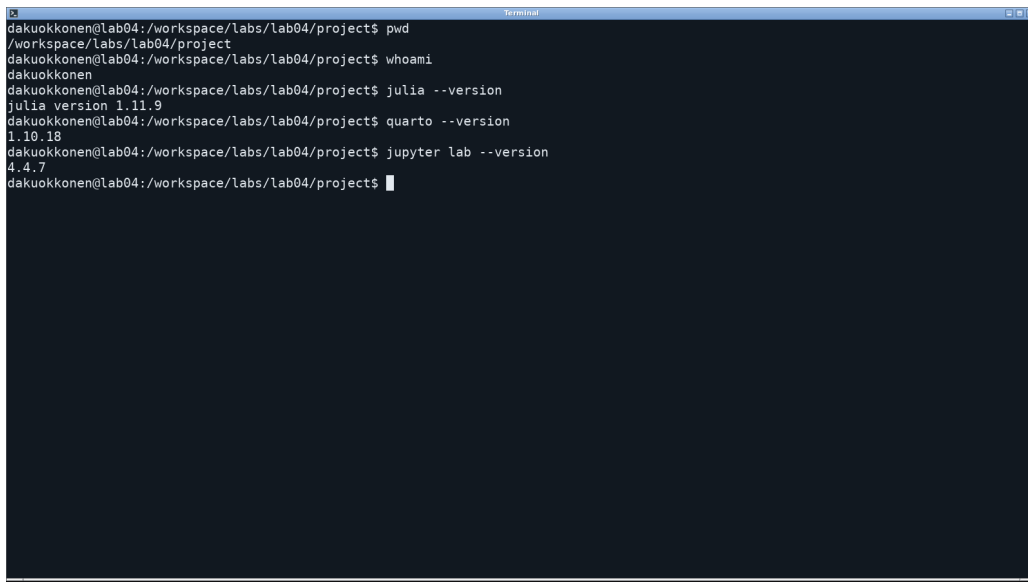
1. Цель и задание
2. Механизмы TCP/Reno и RED
3. Среда и организация проекта
4. Структура модели
5. Базовый эксперимент
6. Исследование числа потоков
7. Производные форматы и Jupyter
8. Проверка и управление версиями
9. Выводы
10. Источники

1. Цель и задание

Цель работы — исследовать совместную динамику алгоритма управления перегрузкой TCP/Reno и активного управления очередью RED в многопоточной сети.

Выполнены следующие задачи:

1. задана агрегированная модель 25 потоков, общего RTT и узкого места;
2. реализованы медленный старт, аддитивное увеличение и мультипликативное уменьшение окна TCP;
3. реализовано экспоненциальное сглаживание очереди и ранний сброс RED;
4. построены графики окна, очереди, вероятности сброса и throughput;
5. исследовано влияние количества конкурирующих потоков;
6. подготовлены данные, блокноты, документация и тесты.



```
dakuokkonen@lab04:/workspace/labs/lab04/project$ pwd
/workspace/labs/lab04/project
dakuokkonen@lab04:/workspace/labs/lab04/project$ whoami
dakuokkonen
dakuokkonen@lab04:/workspace/labs/lab04/project$ julia --version
julia version 1.11.9
dakuokkonen@lab04:/workspace/labs/lab04/project$ quarto --version
1.10.18
dakuokkonen@lab04:/workspace/labs/lab04/project$ jupyter lab --version
4.4.7
dakuokkonen@lab04:/workspace/labs/lab04/project$
```

Рисунок 1: Проверка пользователя, версии Julia и рабочего каталога

2. Механизмы TCP/Reno и RED

TCP/Reno управляет окном перегрузки с wnd . При отсутствии потерь окно растёт: быстро в фазе slow start и примерно на один пакет за RTT в режиме congestion avoidance. Сигнал перегрузки вызывает мультипликативное уменьшение

$$cwnd \leftarrow \max\left(\frac{cwnd}{2}, 1\right). \quad (1)$$

RED принимает решение по сглаженной очереди

$$\dot{q}_{k+1} = (1 - w_q) \dot{q}_k + w_q q_k. \quad (2)$$

Вероятность раннего сброса задаётся кусочно-линейно:

$$p(\dot{q}) = \begin{cases} 0, & \dot{q} < q_{min}, \\ p_{max} \frac{\dot{q} - q_{min}}{q_{max} - q_{min}}, & q_{min} \leq \dot{q} < q_{max}, \\ p_{max}, & \dot{q} \geq q_{max}. \end{cases} \quad (3)$$

Сглаживание (Уравнение 2) ослабляет влияние отдельных всплесков очереди. В данной учебной реализации вероятность насыщается на p_{max} выше q_{max} , а не становится единицей, как в классическом варианте RED. Модель показывает обратную связь между очередью и окнами, но не воспроизводит полный пакетный стек TCP/Reno: нет АСК, таймеров повторной передачи, fast recovery и отдельных линий доступа.

Кусочно-линейная функция вероятности вынесена в общий модуль и непосредственно проверяется модульными тестами.

```
function red_probability(qavg::Real, red::REDParameters)
    qavg < red.qmin && return 0.0
    qavg >= red.qmax && return red.pmax
    return red.pmax * (qavg - red.qmin) / (red.qmax - red.qmin)
end
```

Листинг 1 — Вероятность раннего сброса в src/tcp_red.jl

3. Среда и организация проекта

Расчёты выполнены в Julia 1.11.9. Параметры сети, параметры RED, тип результата и имитационный цикл собраны в src/tcp_red.jl. Базовый сценарий сохраняет траекторию и сводку, а второй сценарий выполняет эксперимент по числу потоков.

Таблица 1: Структура вычислительной части лабораторной работы

Компонент	Назначение
src/tcp_red.jl	структуры параметров и имитация TCP/Reno с RED
scripts/01_tcp_red.jl	базовый эксперимент для 25 потоков
scripts/02_flow_scan.jl	исследование конфигураций от 5 до 40 потоков
scripts/tangle.jl	генерация Julia, QMD и IPYNB
test/runtests.jl	проверки функции RED и ограничений модели

```
dakuokkonen@lab04:/workspace/labs/lab04/project$ find src scripts test -maxdepth 2 -type f | sort
scripts/01_tcp_red.jl
scripts/01_tcp_red/01_tcp_red.jl
scripts/02_flow_scan.jl
scripts/02_flow_scan/02_flow_scan.jl
scripts/setup_environment.jl
scripts/tangle.jl
src/project.jl
src/tcp_red.jl
test/runtests.jl
dakuokkonen@lab04:/workspace/labs/lab04/project$ sed -n '1,100p' Project.toml
name = "project"
uuid = "b43a2660-731f-4e41-9b64-677e6f8c4a2c"
authors = ["dakuokkonen"]
version = "4.0.0"

[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491aa816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Quarto = "d7167be5-f61b-4dc9-b75c-ab62374668c5"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
julia = "1.11"
dakuokkonen@lab04:/workspace/labs/lab04/project$
```

Рисунок 2: Состав Julia-проекта и закреплённые зависимости

3.1 Типы данных и интерфейс модуля

```
module TCPRED
```

```
using Random
using Statistics
```

```
export REDParameters, NetworkParameters, TCPREDResult, red_probability,
simulate_tcp_red
```

```
Base.@kwdef struct REDParameters
```

```
    qmin::Float64 = 75.0
    qmax::Float64 = 150.0
    weight::Float64 = 0.002
    pmax::Float64 = 0.1
    limit::Float64 = 300.0
```

```
end
```

```
Base.@kwdef struct NetworkParameters
```

```
    flows::Int = 25
    capacity_bps::Float64 = 20e6
    packet_bytes::Int = 500
    propagation_rtt::Float64 = 0.118
    duration::Float64 = 30.0
    dt::Float64 = 0.002
```

```
end
```

```
struct TCPREDResult
```

```
    time::Vector{Float64}
    cwnd_first::Vector{Float64}
    cwnd_mean::Vector{Float64}
    queue::Vector{Float64}
    average_queue::Vector{Float64}
    drop_probability::Vector{Float64}
    throughput_mbps::Vector{Float64}
    losses::Int
    delivered_packets::Float64
```

```
end
```

Листинг 2 — Объявления модуля и типы данных из src/tcp_red.jl

4. Структура модели

Учебной постановке соответствует «гантелеобразная» топология: потоки разделяют одно узкое место между маршрутизаторами. В программе она представлена общей очередью, скоростью обслуживания и базовым RTT. Отдельные маршрутизаторы и линии доступа не являются самостоятельными объектами симуляции.

Таблица 2: Параметры базового эксперимента

Параметр	Значение
Число TCP/Reno-потоков	25
Пропускная способность узкого места	20 Мбит/с
Размер пакета	500 байт
Базовый RTT	118 мс
Буфер	300 пакетов
q_{min} / q_{max}	75 / 150 пакетов
w_q	0,002
p_{max}	0,1
Время моделирования	30 с
Шаг времени	0,002 с

В каждый момент скорость потока оценивается как $cwnd/RTT$, где RTT включает задержку накопленной очереди. Принятые пакеты добавляются в буфер, после чего обслуживаются с физической скоростью линии. Для каждого потока независимо разыгрывается сигнал RED.

Структуры NetworkParameters и REDParameters задают параметры с именованными значениями по умолчанию. TCPREDResult содержит синхронизированные векторы окон, очереди, вероятности и throughput. Скорость обслуживания переводится из бит/с в пакеты/с через размер пакета: $20 \cdot 10^6 / (8 \cdot 500) = 5000$. За один шаг линия может обслужить не более десяти пакетов.

```
function simulate_tcp_red(; network::NetworkParameters=NetworkParameters(),
                           red::REDParameters=REDParameters(),
                           seed::Integer=202604)
    network.flows > 0 || throw(ArgumentError("flows must be positive"))
    network.dt > 0 || throw(ArgumentError("dt must be positive"))
    network.duration > network.dt || throw(ArgumentError("duration is too
short"))
    red.qmin < red.qmax <= red.limit || throw(ArgumentError("invalid RED
thresholds"))

    rng = MersenneTwister(seed)
    steps = floor{Int, network.duration / network.dt} + 1
    time = collect(range(0.0; step=network.dt, length=steps))
    cwnd = fill(1.0, network.flows)
    ssthresh = fill(32.0, network.flows)
    queue = 0.0
    qavg = 0.0
    losses = 0
    delivered = 0.0
```

```

packet_bits = 8.0 * network.packet_bytes
service_rate = network.capacity_bps / packet_bits

first_history = zeros(steps)
mean_history = zeros(steps)
queue_history = zeros(steps)
average_history = zeros(steps)
probability_history = zeros(steps)
throughput_history = zeros(steps)

for k in eachindex(time)
    first_history[k] = cwnd[1]
    mean_history[k] = mean(cwnd)
    queue_history[k] = queue
    average_history[k] = qavg
    p = red_probability(qavg, red)
    probability_history[k] = p

    rtt = network.propagation_rtt + queue / service_rate
    sent = cwnd ./ rtt .* network.dt
    arrivals = sum(sent)
    accepted = 0.0

    for i in eachindex(cwnd)
        congestion_probability = 1 - (1 - p)^sent[i]
        if queue >= red.limit || rand(rng) < congestion_probability
            cwnd[i] = max(cwnd[i] / 2, 1.0)
            ssthresh[i] = cwnd[i]
            losses += 1
        else
            accepted += sent[i]
            if cwnd[i] < ssthresh[i]
                cwnd[i] += sent[i]
            else
                cwnd[i] += network.dt / rtt
            end
        end
    end

    queue = min(red.limit, queue + accepted)
    served = min(queue, service_rate * network.dt)
    queue -= served
    delivered += served
    throughput_history[k] = served * packet_bits / network.dt / 1e6
    qavg = (1 - red.weight) * qavg + red.weight * queue
end

return TCPREDResult(time, first_history, mean_history, queue_history,
    average_history, probability_history, throughput_history, losses,
    delivered)
end

```

Листинг 3 — Полная функция моделирования TCP/RED из src/tcp_red.jl

В начале шага сохраняется состояние, вычисляются RTT и объём отправки каждого потока. Величина `sent[i]` вещественная: это агрегированный объём трафика за шаг, а не отдельные дискретные пакеты. Сигнал перегрузки разыгрывается с вероятностью $1 - (1 - p)^{\text{sent}[i]}$. Он уменьшает окно вдвое и меняет порог slow start; без сигнала окно растёт по правилу соответствующей фазы.

Принятый объём добавляется в буфер, ограничивается `red.limit`, затем обслуживается линией. После обслуживания обновляется EWMA. Счётчик `losses` считает сигналы

для потоков за весь прогон, а не число отброшенных пакетов; объём, срезанный ограничением буфера, отдельно в нём не учитывается. Очередь и окна в строке истории относятся к началу шага, throughput — к обслуживанию на этом шаге. Цикл включает и отсчёт при $t=30$ с, то есть выполняет 15001 обновление; для точного баланса по горизонту это соглашение следовало бы отдельно уточнить. В работе анализируются средние сохранённых отсчётов, а не измерения реальной сети.

5. Базовый эксперимент

Базовый сценарий записывает полную траекторию, затем вычисляет средние по отсчётам с $t \geq 10$ с. Первые десять секунд исключены только из усреднения окна, очередей и throughput. Сигналы потери подсчитаны за весь прогон.

```
# # TCP/Reno и очередь RED
#
# **Цель:** исследовать окно перегрузки TCP/Reno и состояние очереди RED в
# многопоточной сети с узким местом 20 Мбит/с.

using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(scrdir("tcp_red.jl"))
using .TCPRED

name = "01_tcp_red"
mkpath(datadir(name)); mkpath(plotsdir(name))
network = NetworkParameters(flows=25, capacity_bps=20e6, packet_bytes=500,
    propagation_rtt=0.118, duration=30.0, dt=0.002)
red = REDParameters(qmin=75.0, qmax=150.0, weight=0.002, pmax=0.1, limit=300.0)
result = simulate_tcp_red(; network, red, seed=202604)

trajectory = DataFrame(time=result.time, cwnd_first=result.cwnd_first,
    cwnd_mean=result.cwnd_mean, queue=result.queue,
    average_queue=result.average_queue,
    drop_probability=result.drop_probability,
    throughput_mbps=result.throughput_mbps)
CSV.write(datadir(name, "trajectory.csv"), trajectory)

warm = trajectory.time .>= 10.0
summary = DataFrame(metric=["mean_cwnd", "mean_queue", "mean_average_queue",
    "mean_throughput_mbps", "loss_events"],
    value=[mean(trajectory.cwnd_mean[warm]), mean(trajectory.queue[warm]),
        mean(trajectory.average_queue[warm]),
        mean(trajectory.throughput_mbps[warm]),
        result.losses])
CSV.write(datadir(name, "summary.csv"), summary)
println("=== TCP/Reno + RED: базовый эксперимент ===")
println("Потоков: $(network.flows), узкое место: $(network.capacity_bps/1e6) Мбит/с")
show(summary; allrows=true, allcols=true); println()

default(fontfamily="DejaVu Sans", linewidth=2.2, framestyle=:box,
    gridalpha=0.22,
    left_margin=5 * Plots.mm)
p1 = plot(trajectory.time, trajectory.cwnd_first; label="первый поток",
    xlabel="Время, с", ylabel="cwnd, пакеты", title="Окно перегрузки TCP/Reno",
    size=(1000, 620), color=:blue)
plot!(p1, trajectory.time, trajectory.cwnd_mean; label="среднее по потокам",
    color=:black)
```

```

savefig(p1, plotsdir(name, "cwnd.png"))

p2 = plot(trajecory.time, trajectory.queue; label="мгновенная очередь",
          xlabel="Время, с", ylabel="Пакеты", title="Очередь RED на узком месте",
          size=(1000,620), color=:orange)
plot!(p2, trajectory.time, trajectory.average_queue; label="EWMA", color=:red)
hline!(p2, [red.qmin, red.qmax]; label=["qmin" "qmax"], linestyle=:dash)
savefig(p2, plotsdir(name, "red_queue.png"))

p3 = plot(trajecory.time, trajectory.drop_probability; label="p(t)",
          xlabel="Время, с", ylabel="Вероятность", title="Вероятность раннего сброса RED",
          size=(1000,620), color=:purple)
savefig(p3, plotsdir(name, "drop_probability.png"))

p4 = plot(trajecory.time, trajectory.throughput_mbps; label="пропускная
способность",
          xlabel="Время, с", ylabel="Мбит/с", title="Использование узкого места",
          size=(1000,620), color=:green)
hline!(p4, [network.capacity_bps/1e6]; label="20 Мбит/с", linestyle=:dash,
color=:black)
savefig(p4, plotsdir(name, "throughput.png"))

# RED реагирует на сглаженную длину очереди, а TCP/Reno отвечает на потери
# мультипликативным уменьшением окна, формируя характерную пилообразную кривую.

```

Листинг 4 — Полный сценарий базового эксперимента из scripts/01_tcp_red.jl

Таблица 3: Результаты базового эксперимента

Показатель	Значение
Среднее окно одного потока	24,5234 пакета
Средняя мгновенная очередь	58,7519 пакета
Средняя EWMA-очередь	56,4558 пакета
Средняя пропускная способность	18,7654 Мбит/с
Число сигналов потери	446

Файлы trajectory.csv и summary.csv сохраняются в data/01_tcp_red. Четыре графика строятся по одной и той же траектории: окно первого потока и среднее окно, мгновенная очередь с EWMA, вероятность сигнала и throughput. Горизонтальные линии показывают пороги RED и ёмкость канала. Число $18,7654/20 \cdot 100\% = 93,83\%$ характеризует использование узкого места на выбранном интервале усреднения.

Окно первого потока демонстрирует характерную пилообразную динамику AIMD: плавный рост чередуется с быстрым делением пополам. Усреднение по потокам сглаживает индивидуальные потери.

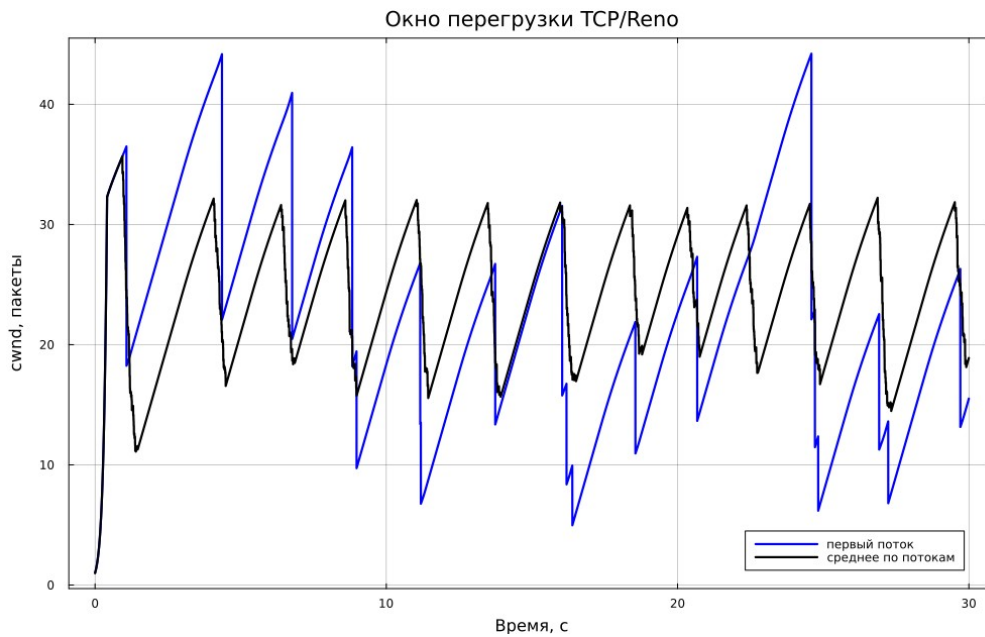


Рисунок 3: Окно перегрузки TCP/Reno

Мгновенная очередь изменяется быстрее EWMA. В базовом режиме среднее значение остаётся ниже q_{min} , однако всплески и накопленная сглаженная оценка периодически активируют ранний сброс.

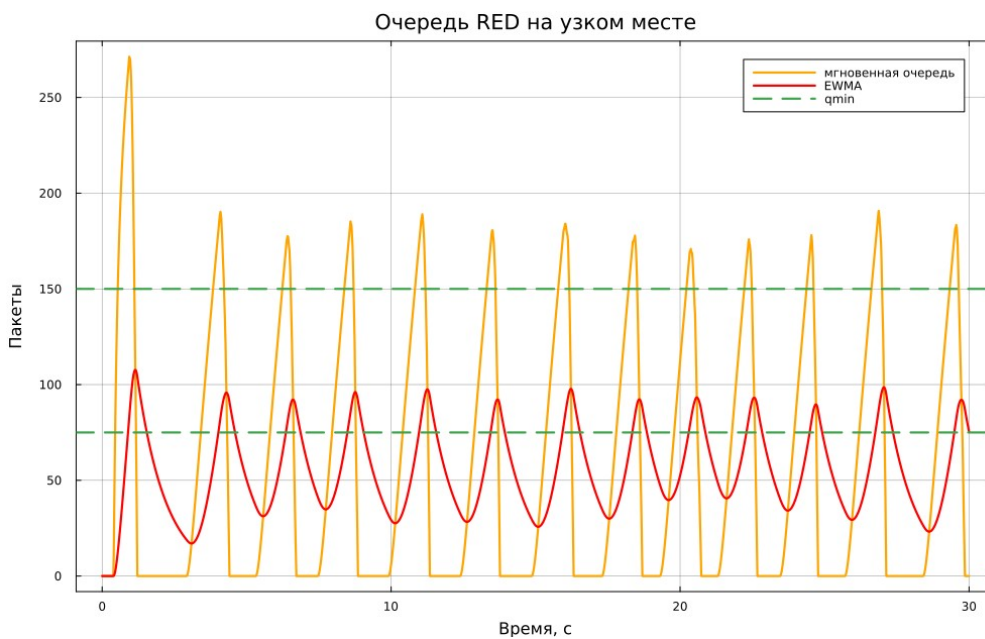


Рисунок 4: Мгновенная и сглаженная очереди RED

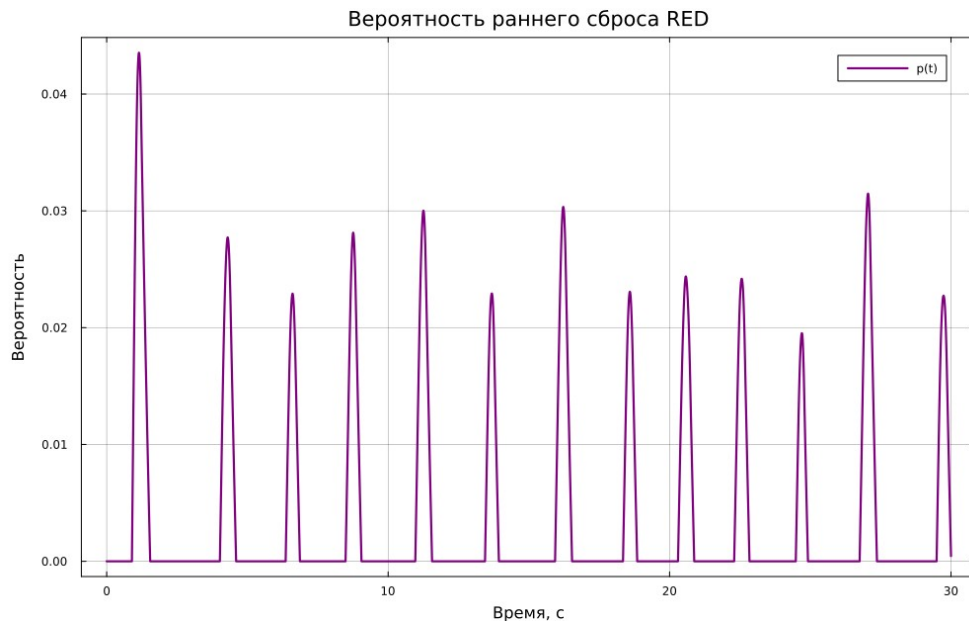


Рисунок 5: Вероятность раннего сброса

Линия большую часть времени используется почти полностью. Краткие провалы связаны с синхронной реакцией части потоков на сигналы перегрузки.

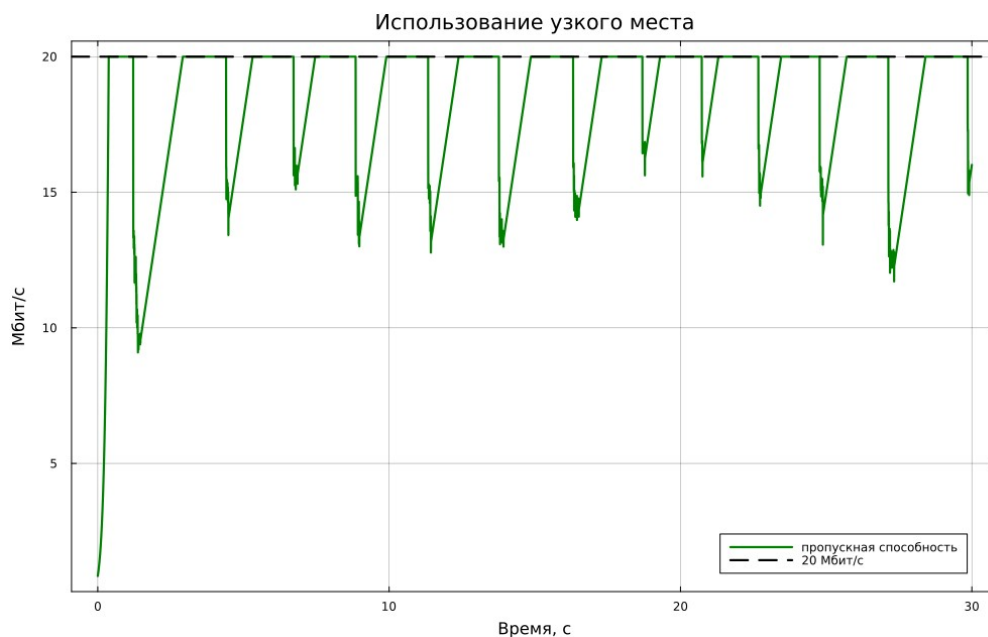


Рисунок 6: Пропускная способность узкого места

6. Исследование числа потоков

Число соединений изменялось от 5 до 40. Для каждого варианта использовалось отдельное фиксированное начальное число генератора.

```
# # Влияние числа ТСП-потоков
#
# **Цель:** оценить загрузку узкого места и состояние RED при изменении числа
```

```

# конкурентных соединений.

using DrWatson
@quickactivate "project"
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(scrdir("tcp_red.jl"))
using .TCPRED

name = "02_flow_scan"
mkpath(datadir(name)); mkpath(plotsdir(name))
red = REDParameters()
rows = NamedTuple[]
for flows in [5, 10, 15, 20, 25, 30, 40]
    net = NetworkParameters(flows=flows, duration=30.0, dt=0.002)
    result = simulate_tcp_red(; network=net, red, seed=202604 + flows)
    warm = result.time .>= 10.0
    push!(rows, (flows=flows,
        throughput_mbps=mean(result.throughput_mbps[warm]),
        mean_queue=mean(result.queue[warm]),
        mean_cwnd=mean(result.cwnd_mean[warm]),
        loss_events=result.losses))
end
scan = DataFrame(rows)
CSV.write(datadir(name, "flow_scan.csv"), scan)
println("=== Сканирование числа TCP-потоков ===")
show(scan; allrows=true, allcols=true); println()

default(fontfamily="DejaVu Sans", linewidth=2.3, framestyle=:box,
gridalpha=0.22,
    left_margin=5 * Plots.mm)
p1 = plot(scan.flows, scan.throughput_mbps; marker=:circle, label="throughput",
    xlabel="Число потоков", ylabel="Мбит/с", title="Загрузка узкого места",
    size=(1000, 620))
hline!(p1, [20.0]; label="ёмкость 20 Мбит/с", linestyle=:dash, color=:black)
savefig(p1, plotsdir(name, "flow_throughput.png"))

p2 = plot(scan.flows, scan.mean_queue; marker=:diamond, label="средняя
очередь",
    xlabel="Число потоков", ylabel="Пакеты", title="Очередь при росте
конкуренции",
    size=(1000, 620), color=:red)
hline!(p2, [red.qmin, red.qmax]; label=["qmin" "qmax"], linestyle=:dash)
savefig(p2, plotsdir(name, "flow_queue.png"))

# С ростом числа потоков узкое место насыщается, а RED удерживает очередь в
# ограниченном диапазоне и сигнализирует TCP до переполнения буфера.

```

Листинг 5 — Полный сценарий эксперимента по числу потоков из scripts/02_flow_scan.jl

Таблица 4: Влияние числа конкурирующих потоков

Потоки	Throughput, Мбит/с	Средняя очередь	Среднее окно	Сигналы
5	19,7009	38,3260	124,0190	10
10	19,6181	45,3713	62,4388	53
15	18,5747	41,1950	39,3207	157
20	18,4500	47,9929	29,6584	277
25	18,4514	53,3248	23,9354	456
30	18,7217	63,9456	20,5946	646
40	18,7592	69,0914	15,6248	1063

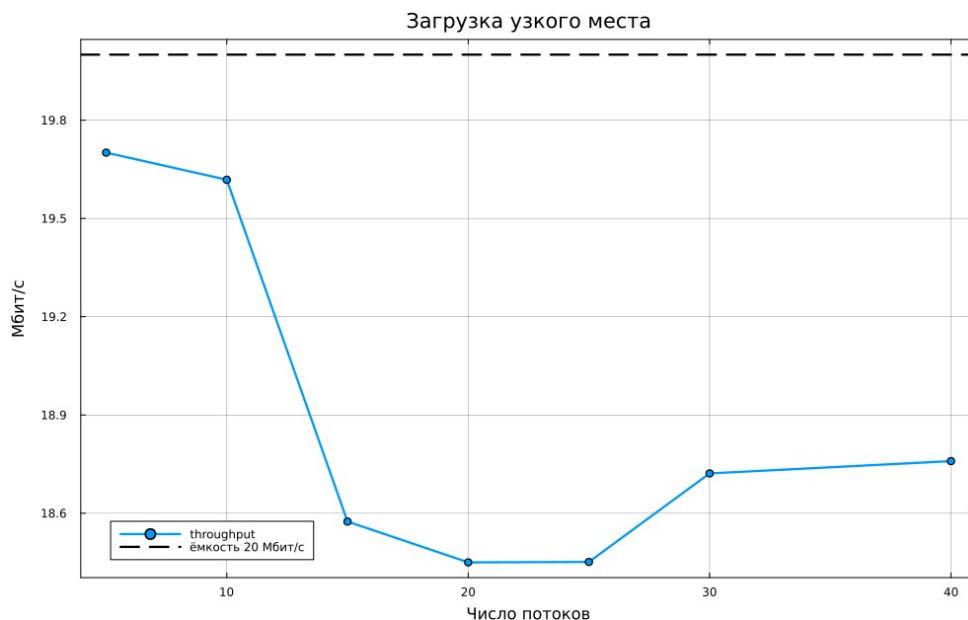


Рисунок 7: Использование узкого места при разном числе потоков

С ростом конкуренции индивидуальное окно уменьшается, а число сигналов потери растёт. Средние очереди в этих прогонах существенно ниже предела 300 пакетов. Сам предел обеспечивается ограничением буфера в коде; без сравнения с режимом без RED нельзя приписать весь эффект только раннему сбросу.

Для каждого числа потоков выполнен один прогон с seed 202604 + flows. Поэтому точка 25 потоков в серии не совпадает с базовым опытом с seed 202604: здесь throughput равен 18,4514 Мбит/с, а число сигналов — 456. Разброс между повторными реализациями и доверительные интервалы не оценивались.

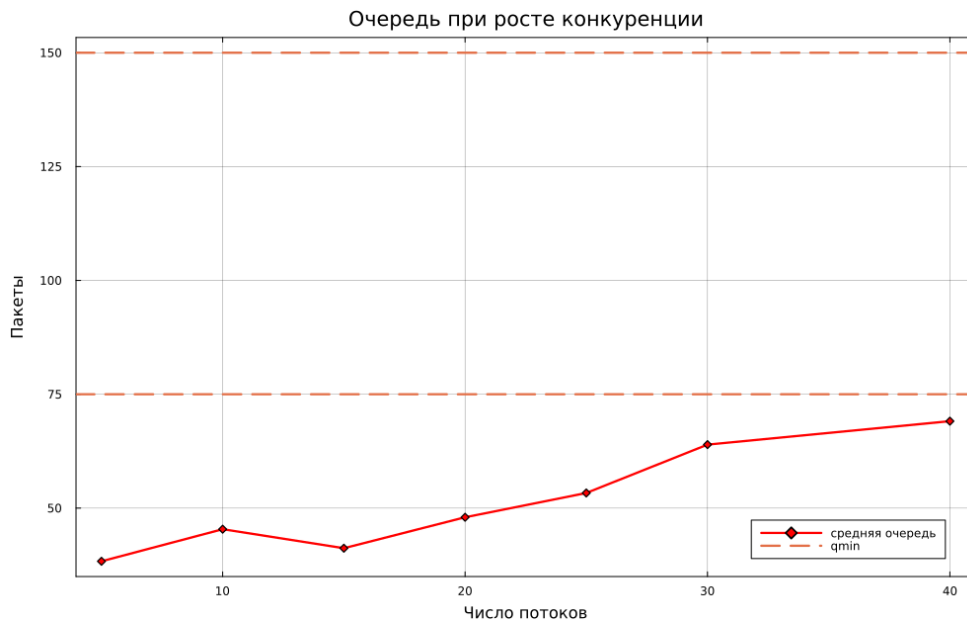


Рисунок 8: Средняя очередь при разном числе потоков

7. Производные форматы и Jupyter

Литературные исходники преобразуются в чистые Julia-файлы, QMD и IPYNB одним генератором. Блокноты выполняются с ядром Julia лабораторной работы и сохраняют фактические сводки базового и параметрического экспериментов.

```
using DrWatson
@quickactivate "project"
using Literate

function generate_formats(source::AbstractString)
    name = splitext(basename(source))[1]
    foreach(mkpath, (scriptsdir(name), projectdir("markdown", name),
projectdir("notebooks", name)))
        Literate.script(source, scriptsdir(name); name=name, credit=false)
        Literate.notebook(source, projectdir("notebooks", name); name=name,
execute=false, credit=false)
        Literate.markdown(source, projectdir("markdown", name); name=name,
flavor=Literate.QuartoFlavor(), credit=false)
    end
    foreach(generate_formats, ARGS)
end
```

Листинг 6 — Генерация производных форматов в scripts/tangle.jl

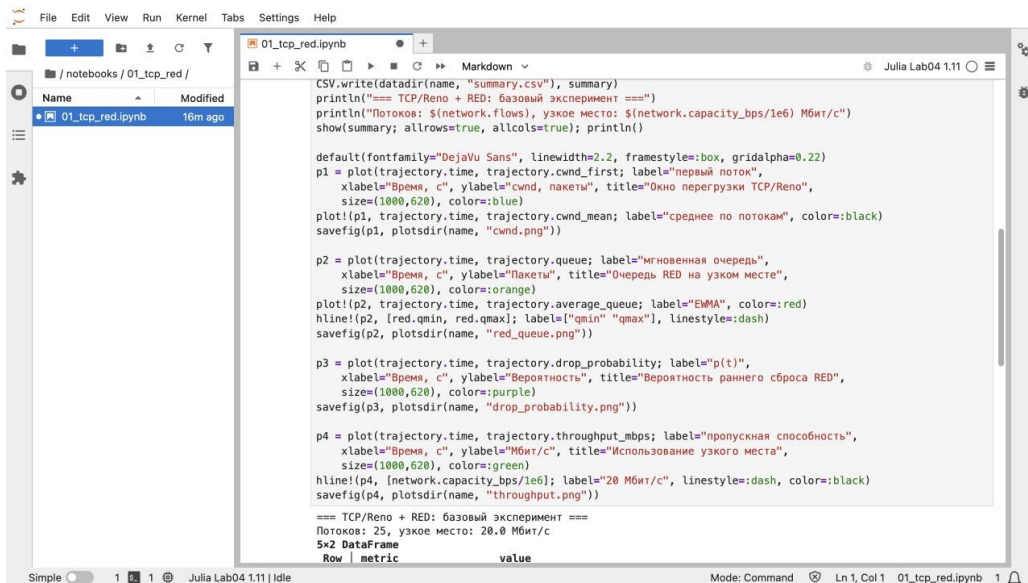


Рисунок 9: Выполненный блокнот базового эксперимента TCP/Reno и RED

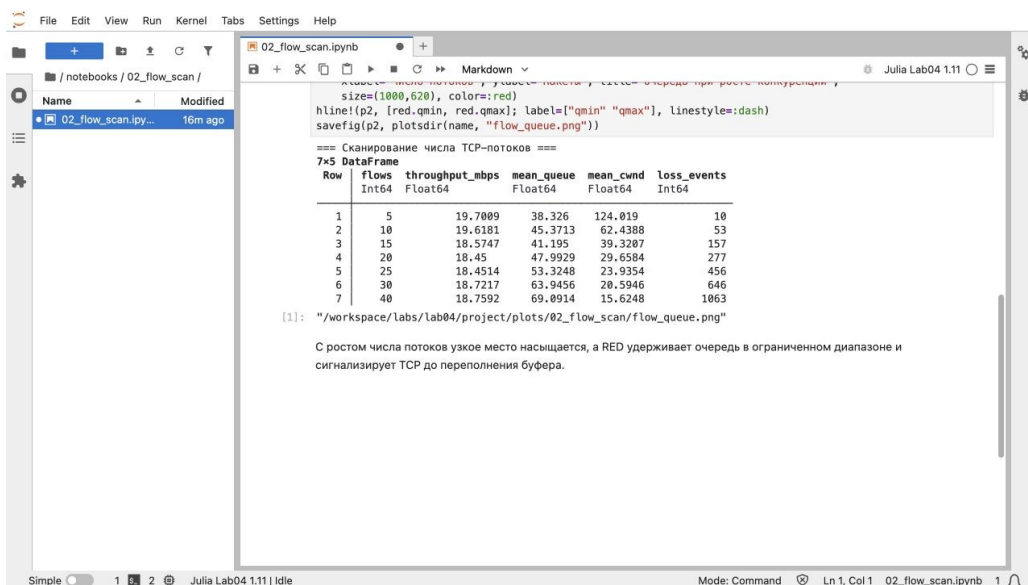


Рисунок 10: Выполненный блокнот эксперимента по числу потоков

8. Проверка и управление версиями

Тесты проверяют кусочно-линейную функцию RED, границы очереди и вероятности, минимальное окно TCP, ограничение throughput физической ёмкостью, наличие доставленных пакетов и появление потерь в нагруженном режиме.

```

using Test
include(joinpath(@__DIR__, "..", "src", "tcp_red.jl"))
using .TCPRED

@testset "RED probability" begin
    red = REDParameters()

```

```

@test red_probability(50.0, red) == 0.0
@test isapprox(red_probability(112.5, red), 0.05)
@test red_probability(150.0, red) == 0.1
end

@testset "TCP/RED simulation" begin
    net = NetworkParameters(flows=25, duration=12.0, dt=0.002)
    red = REDParameters()
    r = simulate_tcp_red(; network=net, red, seed=42)
    @test length(r.time) == length(r.queue)
    @test all(0 .<= r.queue .<= red.limit)
    @test all(r.cwnd_first .>= 1.0)
    @test all(0 .<= r.drop_probability .<= red.pmax)
    @test maximum(r.throughput_mbps) <= net.capacity_bps/1e6 + 1e-9
    @test r.delivered_packets > 0
    @test r.losses > 0
end

```

Листинг 7 — Проверки имитационной модели в test/runtests.jl

Все 10 проверок пройдены: три проверки вероятности RED и семь проверок имитации. Проверяются длины массивов, диапазоны очереди, окна и вероятности, верхняя граница throughput, наличие обслуженного трафика и сигналов перегрузки. Тестовый прогон длится 12 с и использует seed 42. Эти проверки подтверждают перечисленные ограничения реализации, но не проверяют полный протокол TCP.

```

dakuokkonen@lab04:/workspace/labs/lab04/project$ make test
julia --project=. test/runtests.jl
Test Summary: | Pass Total Time
RED probability | 3 3 0.0s
Test Summary: | Pass Total Time
TCP/RED simulation | 7 7 0.7s
dakuokkonen@lab04:/workspace/labs/lab04/project$

```

Рисунок 11: Результат выполнения автоматических проверок

8.1 Воспроизведение расчётов и сборка материалов

Команды выполняются в терминале контейнера из каталога лабораторной.

```

cd /workspace/labs/lab04/project
julia --project=. -e 'using Pkg; Pkg.instantiate()'
make setup
make all
make test
jupyter lab --ip=0.0.0.0 --port=8888 --no-browser

```

Листинг 8 — Восстановление окружения, расчёты и запуск JupyterLab

Цель `tangle` создаёт производные исходники, `run` выполняет оба эксперимента, `notebooks` исполняет IPYNB, `docs` собирает вычислительные HTML-страницы, а `test` запускает проверки общего модуля. В JupyterLab открываются файлы из `notebooks`; ядро Julia устанавливается командой `make setup`.

```
cd /workspace/labs/lab04/report
make all
cd ../presentation
make all
```

Листинг 9 — Сборка отчёта DOCX/PDF и HTML-презентации

Quarto преобразует исходник отчёта в DOCX, LibreOffice создаёт PDF. Презентация хранится в HTML и редактируемом PPTX; `make all` пересобирает HTML из QMD и проверяет наличие обоих форматов. PPTX является отдельным редактируемым документом в принятом оформлении, а не результатом конвертации HTML.

9. Выводы

Построена воспроизводимая модель TCP/Reno с RED. В режиме 25 потоков узкое место обеспечивает в среднем 18,77 Мбит/с, а очередь остаётся значительно ниже физического лимита. Наблюдается ожидаемая пилообразная динамика окна AIMD. Параметрический эксперимент показал уменьшение индивидуального окна и рост числа сигналов перегрузки при увеличении конкуренции. Выводы относятся к агрегированной учебной модели с фиксированным шагом и упрощённым RED; сопоставление с пакетным симулятором и анализ чувствительности к шагу в данной работе не выполнялись.

10. Источники

1. Floyd S., Jacobson V. Random Early Detection Gateways for Congestion Avoidance. *IEEE/ACM Transactions on Networking*, 1993.
2. Stevens W. R. *TCP/IP Illustrated, Volume 1*. Addison-Wesley, 1994.
3. Julia Documentation. <https://docs.julialang.org/>
4. Quarto Documentation. <https://quarto.org/docs/>